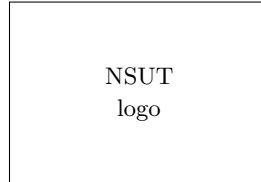


NETAJI SUBHAS UNIVERSITY OF TECHNOLOGY (NSUT)

Department of Computer Science and Engineering

Main Campus, New Delhi



When Should an Alert Threshold Learn?

Admission-Separated Recalibration as a Decision-Time Action
for Time-Series Anomaly Alerting

B.Tech Project-I (Phase-I) Mid-Semester Report

7th Semester, September 2026

Submitted by

Nakul Somani	2023UCS1592
Aman Pandey	2023UCS1595
Pratyush Chaudhary	2023UCS1616

Supervisor

Dr. Vijay Kumar Bohat
Assistant Professor, Department of Computer Science and Engineering

Supervisor's signature

Abstract. Alert thresholds on non-stationary signals are often allowed to learn from recent data. This report asks *what* they should learn from. A threshold that admits only the windows it already judged normal can only fall, and the falling threshold floods the operator with alerts; admitting every window instead absorbs sustained faults into the new normal. We treat recalibration as an explicit decision-time action, chosen alongside ALERT, HOLD and DEFER, whose training data is selected independently of the threshold’s own decisions and kept within a band anchored to the calibration threshold. Eleven policies are replayed on one common score trace per stream, and alert workload, decision coverage, delay and event recall are reported separately. On two SKAB valve streams and seven synthetic stress cases we prove the failure, measure its cost, and find the new controller no worse than a fixed threshold on SKAB but not yet shown to be better on real data. We state the experiment that would confirm or refute the direction.

Contents

1	Introduction	2	6	Simulation Platform and Requirements	9
2	Motivation	3	7	Results and Analysis	9
3	Literature Survey	4	8	Limitations and Threats to Validity	12
4	Problem Statement	5	9	Conclusion	14
5	Objective and Methodology	6	References		14

Our research direction and its seven claims

Recalibrating an alert threshold should be a guarded decision-time action whose training data is chosen neither by that same threshold nor unconditionally, and every alternative should be judged on one common score trace. Tags say how each claim is supported: **PROVED** by proof, **MEASURED** on saved results, **IMPLEMENTED** in tested code, **PROPOSED** not yet tested.

- C1 PROVED** We prove that a threshold which learns only from windows it judged normal can never rise (Proposition 1).
- C2 MEASURED** We measure the cost on two SKAB streams: the same single event is detected as with the fixed threshold, at $16.2\times$ (valve1) and $3.9\times$ (valve2) the alert windows.
- C3 MEASURED** We measure that the scorer, not the policy, limits detection: only 6/100 and 13/98 in-event windows exceed the calibration threshold.
- C4 IMPLEMENTED** We implement recalibration as one of four actions, bounded to $[\theta_0, 4\theta_0]$, and compare it with seven baselines and three ablations on the identical trace.
- C5 MEASURED** On SKAB the controller is identical to the fixed threshold on both streams: a no-harm check, not evidence of benefit.
- C6 MEASURED** Under synthetic stress it cuts false-alert time on a benign step (16 vs 597) and still catches later faults, but fails on a benign ramp and on a fault shaped like a step, as declared in advance.
- C7 PROPOSED** We propose, and do not claim, a benefit on real benign regime changes; a predeclared experiment with three kill criteria decides it (Section 9).

Closest prior work. Each ingredient is already published: the failure in FITNESS [9], abstention in Sun et al. [11] and Perini & Davis [8], workload-based evaluation in Hu [4]. What we did not find in the nine closest papers is their combination on one fixed score trace (Table 2). The strongest case against us is in Section 8.

1 Introduction

An anomaly-alerting system has two parts. A *scorer* turns each window of a sensor signal into a number, the anomaly score; a *policy* turns that score into a decision. The simplest policy raises an alert whenever the score exceeds a fixed threshold θ_0 set on a calibration period. On a signal whose normal level drifts, a fixed threshold goes out of date, so a common remedy is to let the threshold learn from recent scores. This report studies one question inside that remedy:

When should an alert threshold learn, and from which data, if that data may be chosen neither by the threshold’s own decisions nor unconditionally?

We keep the scorer fixed and study the policy only. At every window end our policy chooses one of four actions: ALERT, HOLD (no alert), DEFER (withhold a decision, which lowers decision coverage) or RECALIBRATE (move the threshold). Recalibration is guarded: it uses a buffer of recent scores that is filled without looking at the threshold’s decisions, and it may only move the threshold within $[\theta_0, 4\theta_0]$. All policies replay the identical score trace, so any difference between them is caused by the decision rule and not by a different scorer.

This is a Phase-I report. Its evidence is a development comparison on two SKAB streams [5] and a deterministic synthetic family, not a held-out result. We therefore claim a proved mechanism and honestly scoped measurements, not that any policy wins. Section 2 motivates the question with a worked example; Section 3 positions it against the closest work; Sections 4–6 define the problem, method and platform; Section 7 tests the claims; Sections 8 and 9 give the strongest case against us and the next experiment.

2 Motivation

A worked example: the one-way ratchet. Take a threshold that keeps the last $L = 3$ admitted scores, sets itself to the largest of them, and admits a new score *only if it judged that score normal* ($s \leq \theta$). It starts at $\theta = 10$ with an empty buffer. Table 1 follows six scores.

Table 1: Six steps of a self-referential threshold ($L = 3$, threshold = buffer maximum). A score of 9 is normal at step 1 and an alert at step 6.

Step	Score s	Judged against	Decision	Buffer after	Threshold after
1	9	10	normal, admitted	{9}	9
2	12	9	alert, rejected	{9}	9
3	8	9	normal, admitted	{9, 8}	9
4	7	9	normal, admitted	{9, 8, 7}	9
5	6	9	normal, admitted	{8, 7, 6}	8
6	9	8	alert , rejected	{8, 7, 6}	8

Every admitted score is at most the current threshold, so the buffer maximum can only stay the same or fall. Over a long run the threshold sinks toward the bottom of the normal band, and ordinary fluctuations above it become alerts. The alert flood says nothing about the signal; it is produced by a threshold that learns only from what it already called normal. Section 4 proves this for any score sequence, buffer length and quantile (Proposition 1).

The opposite reflex fails too. Admitting *every* score removes the ratchet but creates the reverse failure. When a sustained fault arrives, its scores fill the buffer, the threshold rises to meet them, and the fault becomes the new normal. One reflex cannot rise; the other cannot tell a fault from a new normal. Our question sits between them.

Why this is not already solved. Drift-adaptive detectors retrain the *model* when drift is detected [6, 12, 14], abstention methods withhold decisions statistically [8, 11], and FITNESS already reports that discarding self-declared anomalies can fail [9]. None of the nine closest papers we checked treats the threshold’s choice of training data as an explicit action and compares that action with its alternatives

on one fixed score trace, with alert workload and decision coverage reported apart. Without that common trace a reported gain can come from a better scorer rather than a better decision, which is the attribution problem this study is built to avoid.

3 Literature Survey

We organise prior work around the five ingredients of our study, because these are what an objector will test one by one: (A) a four-action policy (ALERT, HOLD, DEFER, RECALIBRATE); (B) one frozen score trace shared by all policies; (C) workload and coverage reported beside recall and delay; (D) a chronological source-fit / calibration / replay split; (E) a label-free runtime with labels joined offline. All primary citations come from the seven allowlisted publication platforms.

Drift-adaptive detection. DDADE monitors drift and triggers incremental learning or threshold adjustment [14]; ADAPTS classifies the drift type and reuses models from a bounded pool [6]; CDDIA interprets drift in normal data and retrains the detector [12]. All three change the *model*. We freeze the scorer and change only the action taken at a window end.

Abstention and rejection. Sun et al. output $\{0, 1, \text{abstain}\}$ on piecewise-stationary streams with confidence-sequence guarantees [11], the strongest threat to our DEFER. Perini & Davis reject uncertain points with a constant threshold on a stability metric [8]. Both abstain as a statistical decision. Our DEFER is an operational action whose cost is charged to decision coverage and reported beside workload. We re-implemented Sun et al.’s Algorithm 1 as a baseline, so this comparison is measured rather than argued.

Label-free selection and workload-based evaluation. SEAD selects among detectors online and emits a new combined score [10], the opposite of ingredient (B); it could sit upstream as our scorer. Hu makes alert episodes and workload the object of evaluation [4] (in press); we adopt that frame and add a controller to it. Dynamic-XY learns alert suppression from historical *alerts* [1], not from a score trace with explicit actions.

The failure mode is already published. FITNESS states that “MEMSTREAM discards a sample if it is predicted as an anomaly at the time of scoring ... such techniques can fail to distinguish between anomalies and abrupt changes” [9]. We therefore *do not* claim the phenomenon. We claim only the deterministic monotone-ratchet statement for nearest-rank quantile thresholds with decision-gated admission (Proposition 1). VUS and TSB-AD show that ranking metrics are functions of the score trace alone [2, 7], which is why policies on one trace are compared by alert behaviour, not by ranking metrics. Conformal and change-point thresholding are adjacent context [3, 13].

Where the gap is. Table 2 maps the ingredients to the nine closest papers. Every action in (A), and ingredients (C) and (E), are already occupied; DEFER is occupied as a concept by Sun et al. and Perini & Davis. The only open cell is (B): no checked method holds the score fixed and compares *action policies* on it.

Table 2: Component-occupation matrix: ✓ occupies, p partial (adjacent), — not found in the nine checked papers. Threats: 1 DDADE, 2 ADAPTS, 3 CDDIA, 4 Sun 2024, 5 Perini & Davis, 6 SEAD, 7 Dynamic-XY, 8 Hu, 9 FITNESS.

Our component	1	2	3	4	5	6	7	8	9
A. alert action	✓	✓	✓	✓	✓	✓	✓	✓	✓
A. hold / suppress action	p	✓	p	—	—	—	✓	—	—
A. defer / abstain action	—	—	—	✓	✓	—	—	—	—
A. recalibrate action	✓	✓	✓	p	—	—	—	✓	✓
B. common frozen trace across policies	—	—	—	—	—	—	—	—	—
C. workload + coverage reporting	—	—	—	—	p	—	✓	✓	—
D. chrono. source/calib/replay split	—	—	p	—	—	—	—	✓	—
E. label-free runtime (labels offline)	✓	✓	✓	✓	✓	✓	✓	—	✓

Honest scope. What we did not find is recalibration treated as a guarded decision-time action whose data is chosen neither by the threshold being updated nor unconditionally, compared with its alternatives on one fixed score trace with workload, coverage, delay and recall reported separately. This is a bounded search over nine papers, not a claim that no such method exists. If the direction holds, that combination and its clean attribution are our contribution.

4 Problem Statement

Setting. A fixed scorer, fit on source data only, emits one score s_t per window at window end $t = 1, 2, \dots$ (window length 4, stride 4, incomplete windows dropped). Every decision at time t may use only scores up to t . Labels are never read at runtime; they are joined offline after all predictions are saved. The notation is:

- s_t : the score of the window ending at t ; the sequence is the *score trace*.
- θ_t : the threshold in force at t ; an alert is raised if $s_t > \theta_t$.
- θ_0 : the calibration threshold, the nearest-rank 0.95 quantile of the calibration-window scores. It is frozen and serves as the *anchor*.
- R : a buffer of recent scores; $q(R)$ is its nearest-rank 0.95 quantile (the $\lceil 0.95 |R| \rceil$ -th smallest) and $\text{med}(R)$ its median.

What we measure. For each stream and policy (an *arm*) we count *alert windows* and *alert episodes* (runs of consecutive alerts), *false episodes* (episodes that touch no labelled event), *event recall* (events touched by at least one alert), *non-event alert duration* (alerted samples outside events, our main measure of false-alert workload) and the *in-event alerted fraction*. *Decision coverage* is the share of windows that received an alert-capable decision rather than a DEFER. The full tables also give episode precision, mean delay and episodes per 1,000 decisions. Every value shows its denominator, and durations and delays are in sample-index units. Because all arms replay the same trace, ranking metrics such as VUS-PR are identical by construction and are not reported per arm.

The self-referential ratchet. The rolling baseline admits a score to R only when the current threshold judged it normal, then sets the threshold to $q(R)$. We claim this threshold can never rise.

Proposition 1 (Self-referential admission ratchets). *Update a threshold as follows: admit the current score s to a buffer of capacity $L \geq 1$ only if $s \leq \theta$, drop the oldest element when over capacity, and set*

the new threshold to the nearest-rank q -quantile of the buffer, $q \in (0, 1]$. Then for every score sequence, $\theta_{t+1} \leq \theta_t$ for all t .

Proof. Let $n = |R|$ and $k = \lceil qn \rceil$, so θ_t is the k -th smallest element of R and at least k elements of R are $\leq \theta_t$. If s is not admitted, R is unchanged and $\theta_{t+1} = \theta_t$. If s is admitted, then $s \leq \theta_t$, and there are two cases. *Buffer not full:* the new buffer has $n + 1$ elements, at least $k + 1$ of them $\leq \theta_t$, and its rank is $k' = \lceil q(n + 1) \rceil \leq k + 1$, so its k' -th smallest element is $\leq \theta_t$. *Buffer full:* the oldest element is dropped, the size and rank stay n and k , and at least $(k + 1) - 1 = k$ elements $\leq \theta_t$ remain, so the k -th smallest is $\leq \theta_t$. From an empty buffer the first admitted score satisfies $s \leq \theta_0$ and sets $\theta = s$. In every case $\theta_{t+1} \leq \theta_t$. $\square$

The proposition explains the rolling baseline’s alert volume without looking at any data. Admitting every score has no such bound and follows whatever arrives, including a sustained fault. The problem is therefore to design an admission rule that is *separated* from the threshold’s own decisions and *anchored* to θ_0 , and to measure what it costs in coverage and delay.

5 Objective and Methodology

Objective. Test whether separating a threshold’s recalibration data from that threshold’s own decisions, under an anchored guard with explicit deferral, avoids both the ratchet and fault absorption, and measure what it costs in coverage and delay, on one common score trace against every baseline.

Pipeline. Each stream is split in time order into source-fit, calibration and replay segments (Figure 1). The scorer is fit on the source segment and frozen, calibration gives θ_0 , and replay gives the one score trace that every arm receives. Labels are joined offline, after predictions are saved.

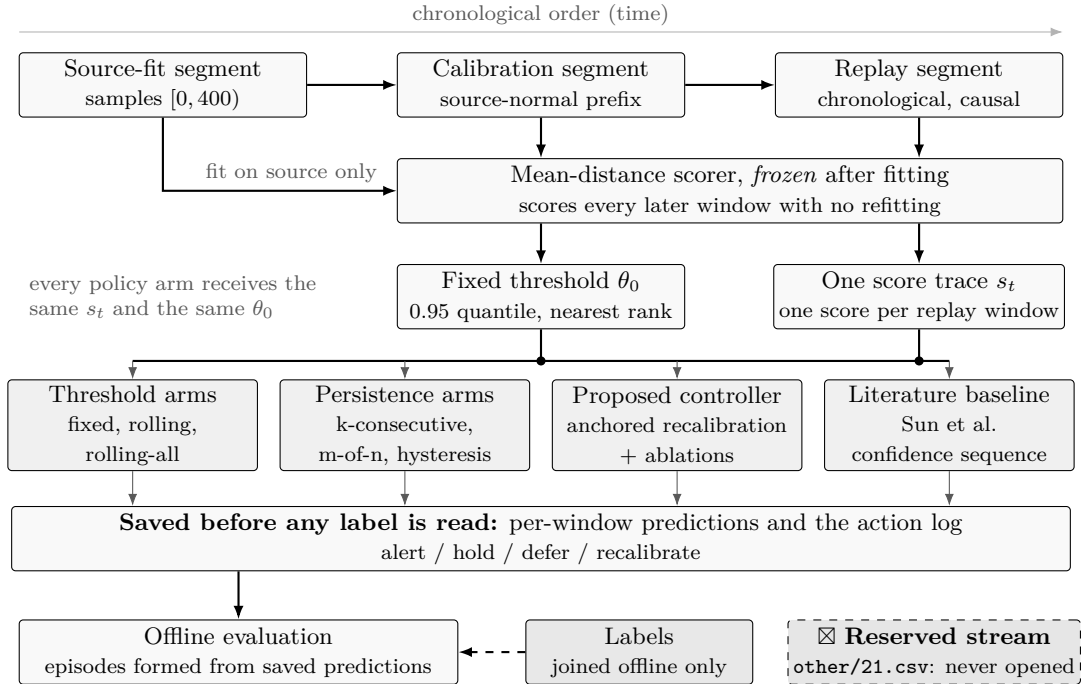


Figure 1: Chronological split and causal data flow. One frozen scorer produces one score trace, which every policy arm receives unchanged; outputs are saved before labels are joined offline. The reserved stream is never opened.

C4 IMPLEMENTED *We implement recalibration as one guarded action among four.*

The proposed controller (class `AnchoredRecalibrationPolicy`) keeps a threshold θ starting at θ_0 , a mode (NORMAL, SUSPECT or ESCALATED), a defer counter c , and a buffer R of the last L scores of *every* window, so admission never depends on a decision. Listing 1 is the frozen rule and Table 3 summarises it.

```

append s to R (drop oldest beyond L)
full      = |R| == L
elevated = full and median(R) > theta
if mode == escalated:
    if elevated: return ALERT if s > theta else HOLD      # stay escalated, no recalibration
    mode = normal                                          # elevation over; fall through
if elevated:
    cand = quantile(R)
    stable = max(R) <= rho * median(R)
    if stable and cand <= kappa * theta0:
        theta = max(theta0, cand); mode = normal; c = 0 # action RECALIBRATE (up)
        return (ALERT if s > theta else normal), action=RECALIBRATE
    if c < D: c += 1; mode = suspect; return DEFER        # guard failed: withhold
    mode = escalated; c = 0; return ALERT if s > theta else HOLD
# not elevated
if mode == suspect: mode = normal; c = 0
if full and theta > theta0 and quantile(R) < theta:
    theta = max(theta0, quantile(R))                      # action RECALIBRATE (return to anchor)
return ALERT if s > theta else HOLD

```

Listing 1: The frozen controller, transcribed exactly from the protocol. `quantile` is nearest rank; `median` is the ordinary median.

Table 3: When the controller takes each action. “Sustained rise” means R is full and $\text{med}(R) > \theta$.

Situation at a window end	Action and mode
No sustained rise	HOLD, or ALERT if $s > \theta$; mode normal
Sustained rise, stable ($\max R \leq \rho \text{med}(R)$) and $q(R) \leq \kappa\theta_0$	RECALIBRATE to $\max(\theta_0, q(R))$; mode normal
Sustained rise, guard fails, fewer than D deferrals so far	DEFER; mode suspect
Sustained rise, guard still failing after D deferrals	escalate: ALERT if $s > \theta$ else HOLD; no recalibration while the rise lasts
Rise over, $\theta > \theta_0$ and $q(R) < \theta$	RECALIBRATE back to $\max(\theta_0, q(R))$

Parameters were fixed before any outcome was seen. $L = 10$ and $q = 0.95$ are the existing rolling baseline’s values, and the defer limit is $D = L = 10$. The cap $\kappa = 4$ declares the largest benign rise an operator accepts (four times θ_0), and the stability bound $\rho = 2$ declares that no window in a buffer may exceed twice its median. Both are operating assumptions, not estimates; on the SKAB streams the guard is never reached, so no SKAB outcome could have informed them. Figure 2 contrasts the three admission rules schematically.

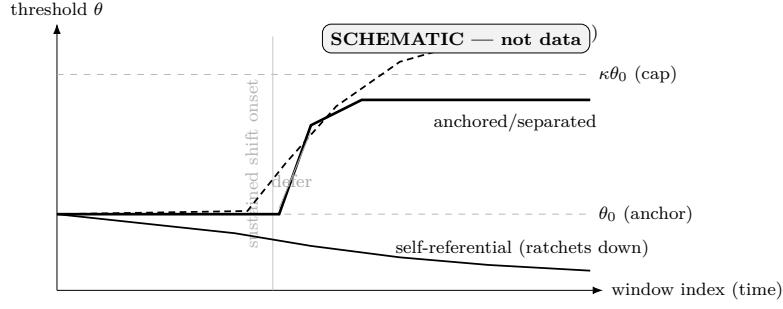


Figure 2: Schematic, not data. Self-referential admission ratchets the threshold down; unconditional admission follows a sustained fault up; anchored, separated admission stays in $[\theta_0, \kappa\theta_0]$ and defers through an unstable transition.

Policy arms. Table 4 lists the eleven arms. The five previously audited arms reproduce their corrected baseline rows exactly, which serves as a regression gate.

Table 4: Policy arms. Every arm replays the identical score trace of its stream.

Arm	Role
fixed_threshold	anchor only (existing)
rolling_threshold (normal-only)	self-referential admission, $L=10$, $q=0.95$ (existing)
rolling_threshold_all	admits every window, no guard (existing)
k_consecutive ($k=3$), m_of_n (3/5), hysteresis	persistence and latch rules (existing)
anchored_recalibration	proposed controller (new)
anchored_no_cap / no_stability / no_defer	ablations: $\kappa=\infty$ / $\rho=\infty$ / $D=0$
sun_confidence_sequence	Sun et al. (ICML 2024) Algorithm 1, re-implemented

Synthetic stress family. Table 5 defines seven deterministic stress cases on a period-16 signal of length 1600 (input `synthetic_periodic_v1`), using the same windows, scorer, calibration and arms. Offsets apply over half-open ranges; a spiky fault adds +6.0 on alternating 4-sample windows. These cases test the mechanism; they are not evidence of real-world benefit.

Table 5: Synthetic stress family: what each case adds and what it tests.

ID	Offsets / fault	Tests
Y1	+2.0 on [800, 1600); no event	adapting to a new normal
Y2	+0.25 at each of 800, 810, ..., 870 up to +2.0; no event	an unstable transition (defer)
Y3	Y1 plus a spiky fault on [1200, 1300)	catching a fault after adapting
Y4	+2.0 on [800, 1600), labelled as an event	the declared limit (H4)
Y5	spiky fault on [800, 900)	onset alert, then the cost of deferring
Y6	+5.0 on [800, 1600); no event	the cost of the cap κ
Y7	+2.0 on [800, 1100); spiky fault on [1300, 1400)	returning to the anchor

Hypotheses. Table 6 fixes each hypothesis and the result that would refute it. H2 is a no-harm check, not a test of benefit: before freezing we saw that no 10-window block of either SKAB stream has a median above θ_0 , so the controller was predicted to act exactly like the fixed threshold there. Benefit is tested only in H3 and in the next experiment (Section 9).

Table 6: Hypotheses, fixed before the study ran. All tests use common score traces.

ID	Hypothesis	Refuted if
H1	Normal-only rolling recalibration is a one-way ratchet, and its SKAB alert volume is a result of that.	The normal-only rolling threshold ever increases.
H2	The controller keeps event recall 1/1 on valve1 and valve2 with false episodes $\leq$ fixed (no harm).	Recall $< 1/1$ on either stream, or more false episodes than fixed.
H3	On benign synthetic shifts the controller cuts non-event alert duration versus fixed and normal-only rolling, and still alerts on spiky faults after a shift.	No cut on Y1/Y2, or the Y3/Y7 faults are missed.
H4	No label-free rule can separate a benign shift from a fault of identical shape, so the controller absorbs Y4 after onset.	Declared limit: reported either way, not a refutation target.

6 Simulation Platform and Requirements

Software. The platform is a Python 3.12 package that uses only the standard library, with `unittest` for its tests. It provides reusable loaders, chronological splits, a scoring interface, fixed-quantile calibration, the policy arms and an offline evaluator. Runs save per-window predictions and an action log, and every reported row is recomputed by an independent script (`recompute_evaluations.py`) before it enters this report. Execution is deterministic: scoring and policies use no randomness, and the one randomized property test seeds its own generator. No GPU is needed; each run records its elapsed time and peak Python memory as its resource scope. The report and its figures build with a basic $\text{\LaTeX}$ Live 2026 installation.

Data. **MEASURED** The development data are two SKAB streams, `valve1/0.csv` and `valve2/0.csv` [5], scored on the single **Current** channel. Each contains exactly one labelled event (valve1 [574, 975), valve2 [562, 956)), so recall is measured over one event per stream. Origin and licence are not independently verified (valve1’s acquisition URL is unknown; valve2’s provenance comes from an earlier manifest), so all results are development-only. The stream `SKAB/other/21.csv` is named as reserved in the manifest, but the authority and recipe for that reservation are disputed; it was never opened and no claim rests on it.

7 Results and Analysis

All numbers come from the saved study 20260929-policy-study-v2, which an independent script recomputes with zero mismatches over 99 runs. Within a stream every arm replays a byte-identical score trace, so differences come from decision rules alone. Durations and delays are in sample-index units.

7.1 The ratchet and its cost (C1, C2; H1)

C1 **PROVED** **MEASURED** *We prove the ratchet and observe it in the saved traces.*

The normal-only rolling threshold records 0 increases in 143 (valve1) and 140 (valve2) replay steps. It falls from 1.309 to 0.258 on valve1 and from 1.126 to 0.406 on valve2, below the median replay score (Figure 3), exactly as Proposition 1 predicts.

C2 **MEASURED** *We measure what the ratchet costs.*

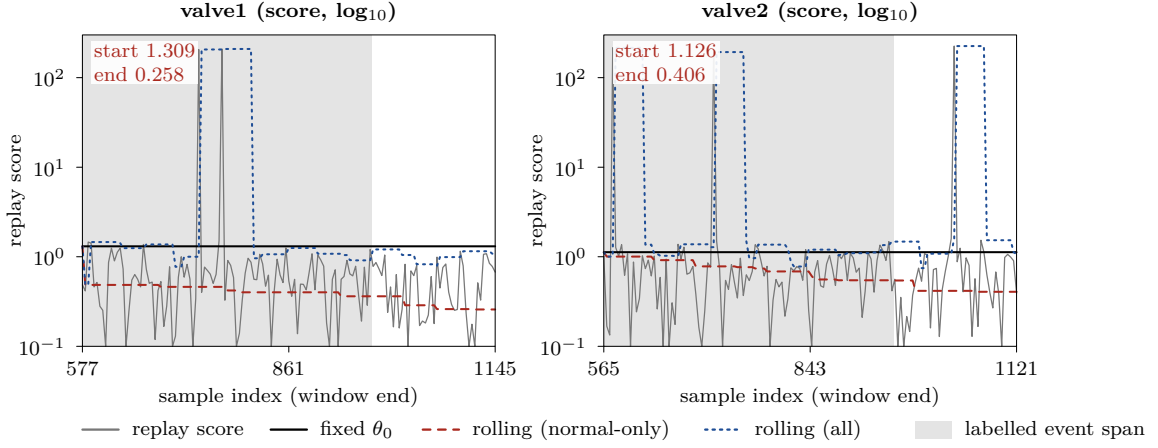


Figure 3: Replay score (grey, $\log_{10}$) on valve1 (left) and valve2 (right), with the fixed threshold θ_0 (black), the normal-only rolling threshold (red dashed) and the admit-all rolling threshold (blue dotted); the labelled event is shaded. Start and end threshold values are printed inside each panel. The normal-only threshold only ever falls.

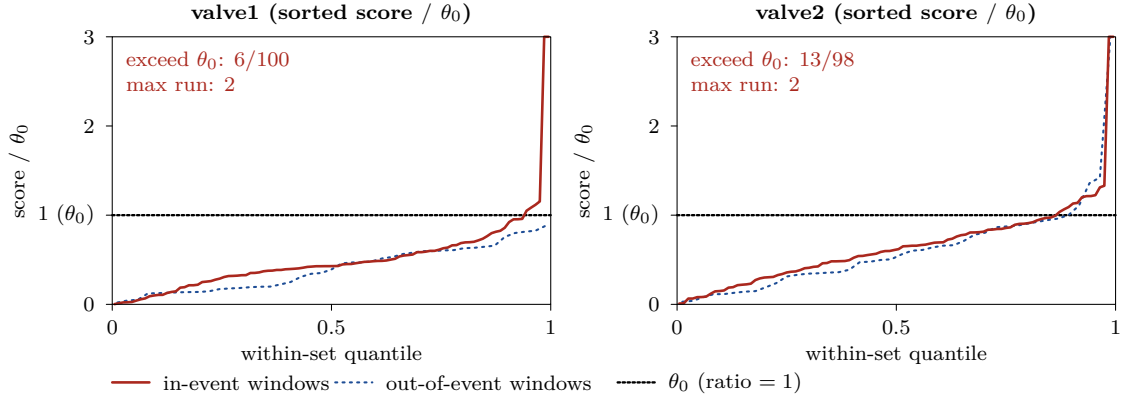


Figure 4: Scores sorted within each set, as a ratio to θ_0 : in-event windows (red solid), out-of-event windows (blue dotted), θ_0 at ratio 1 (black dotted). Few in-event windows cross the line. The y axis is clipped at $3\theta_0$; 2 (valve1) and 3 (valve2) windows lie above the clip.

Rolling (normal-only) detects the same single event as the fixed threshold, but with $16.2\times$ the alert windows on valve1 (97 vs 6) and $3.9\times$ on valve2 (70 vs 18) (Table 7). The extra alerts come from the falling threshold, not from extra anomalies.

7.2 The scorer, not the policy, limits detection (C3)

C3 MEASURED *We measure that no alert rule can recover what the scorer does not separate.*

Inside the labelled event only 6/100 (valve1) and 13/98 (valve2) replay windows exceed θ_0 , and the longest run above it is 2 windows on both streams (Figure 4). Persistence rules therefore fall short of their own requirement: $k_consecutive$ ($k = 3$) misses both events and m_of_n (3/5) misses valve1. Each event also covers about 70% of the replay decisions, so a single alert anywhere inside it scores full recall. On SKAB, recall of 1/1 carries little information; workload, false episodes and the in-event alerted fraction carry it.

7.3 No harm on SKAB, and no evidence of benefit (C5; H2)

C5 MEASURED *We measure that the controller is identical to the fixed threshold on both SKAB streams.*

On valve1 both give 6 alert windows, 4 episodes, 0 false episodes and recall 1/1; on valve2 both

Table 7: All 11 arms on the two SKAB development streams, one common score trace per stream. Recall and coverage are shown as num/den. Calibration threshold θ_0 : valve1 1.309 (43 windows), valve2 1.126 (40 windows). “In-ev. alert” is the in-event alerted fraction; “–” marks an undefined value (mean delay when an arm raises no in-event alert).

Stream	Arm	Alert win.	Epis.	Recall n/d	False epis.	Mean delay	Non-ev. dur.	In-ev. alert	Cover. n/d	Def.	Recal.
valve1	fixed	6	4	1/1	0	11.0	0	0.060	143/143	0	0
	rolling (n.o.)	97	27	1/1	8	11.0	109	0.693	143/143	0	0
	rolling (all)	10	10	1/1	2	11.0	10	0.075	143/143	0	0
	k-consec.	0	0	0/1	0	–	0	0.000	143/143	0	0
	m-of-n	0	0	0/1	0	–	0	0.000	143/143	0	0
	hysteresis	6	4	1/1	0	11.0	0	0.060	143/143	0	0
	anchored	6	4	1/1	0	11.0	0	0.060	143/143	0	0
	no-cap	6	4	1/1	0	11.0	0	0.060	143/143	0	0
	no-stability	6	4	1/1	0	11.0	0	0.060	143/143	0	0
	no-defer	6	4	1/1	0	11.0	0	0.060	143/143	0	0
	Sun	3	3	1/1	0	11.0	0	0.030	54/143	89	0
valve2	fixed	18	12	1/1	3	15.0	20	0.132	140/140	0	0
	rolling (n.o.)	70	29	1/1	9	15.0	104	0.447	140/140	0	0
	rolling (all)	13	12	1/1	2	15.0	12	0.102	140/140	0	0
	k-consec.	0	0	0/1	0	–	0	0.000	140/140	0	0
	m-of-n	6	3	1/1	1	27.0	9	0.038	140/140	0	0
	hysteresis	21	9	1/1	3	15.0	20	0.162	140/140	0	0
	anchored	18	12	1/1	3	15.0	20	0.132	140/140	0	0
	no-cap	18	12	1/1	3	15.0	20	0.132	140/140	0	0
	no-stability	18	12	1/1	3	15.0	20	0.132	140/140	0	0
	no-defer	18	12	1/1	3	15.0	20	0.132	140/140	0	0
	Sun	2	2	1/1	1	15.0	4	0.010	57/140	83	0

give 18 windows, 12 episodes, 3 false episodes and recall 1/1. The controller makes 0 deferrals and 0 recalibrations, because its guard is never reached. H2 therefore holds as a no-harm check; it says nothing about benefit. Fixed and hysteresis detect each event at low alert volume. The Sun et al. arm decides on only 54/143 (valve1) and 57/140 (valve2) windows and reaches its low workload by abstaining; it is the only arm that defers on SKAB.

7.4 Mechanism and declared failures under synthetic stress (C6; H3, H4)

C6 MEASURED *We measure where the mechanism works and where it fails.*

Table 8 and Figure 5 give the synthetic family.

- *Works on a benign step (Y1).* Non-event alert duration is 16, against 597 for fixed and 917 for normal-only rolling, at full coverage 260/260, with one recalibration and no deferral.
- *Still catches later faults (Y3, Y7).* After adapting, it alerts the spiky fault with recall 1/1 and in-event alerted fraction 0.29 on both.
- *Fails on a benign ramp (Y2).* It recalibrates once during the ramp and then locks into periodic single-window alerts: 204 over 50 episodes at coverage 257/260 (3 deferrals), against 577 for fixed.
- *Absorbs a fault shaped like a step (Y4), as declared.* Its in-event alerted fraction is 0.020 against 0.746 for fixed. No label-free rule can separate a benign shift from a fault of the same shape (H4).

Which guard component matters. Table 9 removes one guard component at a time, on the same trace. The cap κ matters most on Y6, a step larger than the cap: the full controller cannot follow it, so it defers 10 times, escalates, and keeps alerting (non-event alert duration 757); without the cap it follows the step and the duration falls to 16. Removing deferral costs most on the Y2 ramp, where non-event alert duration rises by 373. These are measured effects of fixed components, not tuning.

Table 8: Synthetic cases Y1–Y7, five arms (fixed, admit-all rolling, anchored, no-cap, Sun). “Non-ev. dur.” is non-event alert duration; “In-ev. alert” is the in-event alerted fraction; “Def./Rec.” is deferrals/recalibrations; “–” marks a value undefined because the case has no labelled event.

Scen.	Arm	Non-ev. dur.	In-ev. alert	Recall n/d	Cover. n/d	Def./Rec.
Y1	fixed	597	–	–	260/260	0/0
	rolling (all)	12	–	–	260/260	0/0
	anchored	16	–	–	260/260	0/1
	no-cap	16	–	–	260/260	0/1
	Sun	8	–	–	22/260	238/0
Y2	fixed	577	–	–	260/260	0/0
	rolling (all)	28	–	–	260/260	0/0
	anchored	204	–	–	257/260	3/1
	no-cap	204	–	–	257/260	3/1
	Sun	24	–	–	23/260	237/0
Y3	fixed	521	1.000	1/1	260/260	0/0
	rolling (all)	12	0.040	1/1	260/260	0/0
	anchored	19	0.290	1/1	250/260	10/1
	no-cap	16	0.160	1/1	260/260	0/3
	Sun	8	0.040	1/1	22/260	238/0
Y4	fixed	0	0.746	1/1	260/260	0/0
	rolling (all)	4	0.010	1/1	260/260	0/0
	anchored	0	0.020	1/1	260/260	0/1
	no-cap	0	0.020	1/1	260/260	0/1
	Sun	0	0.010	1/1	22/260	238/0
Y5	fixed	3	0.490	1/1	260/260	0/0
	rolling (all)	4	0.040	1/1	260/260	0/0
	anchored	3	0.290	1/1	250/260	10/0
	no-cap	0	0.160	1/1	260/260	0/2
	Sun	0	0.040	1/1	90/260	170/0
Y6	fixed	797	–	–	260/260	0/0
	rolling (all)	12	–	–	260/260	0/0
	anchored	757	–	–	250/260	10/0
	no-cap	16	–	–	259/260	1/1
	Sun	8	–	–	22/260	238/0
Y7	fixed	227	0.490	1/1	260/260	0/0
	rolling (all)	12	0.040	1/1	260/260	0/0
	anchored	19	0.290	1/1	250/260	10/2
	no-cap	16	0.160	1/1	260/260	0/4
	Sun	8	0.040	1/1	60/260	200/0

7.5 Workload against coverage

MEASURED Figure 6 places every arm by decision coverage and non-event alert duration. On SKAB nearly all arms sit at full coverage, so only the vertical spread of workload separates them. On the benign cases the Sun arm is quiet only because it decides on 22/260 (Y1) and 23/260 (Y2) windows. Section 8 reads this trade-off against us.

8 Limitations and Threats to Validity

Threats. *One event per stream:* each SKAB stream has a single labelled event covering about 70% of replay, so recall saturates (Section 7.2). *Scorer limit:* few in-event windows exceed θ_0 , which bounds every policy. *Univariate scope:* only the **Current** channel is scored. *Provenance:* dataset origin and

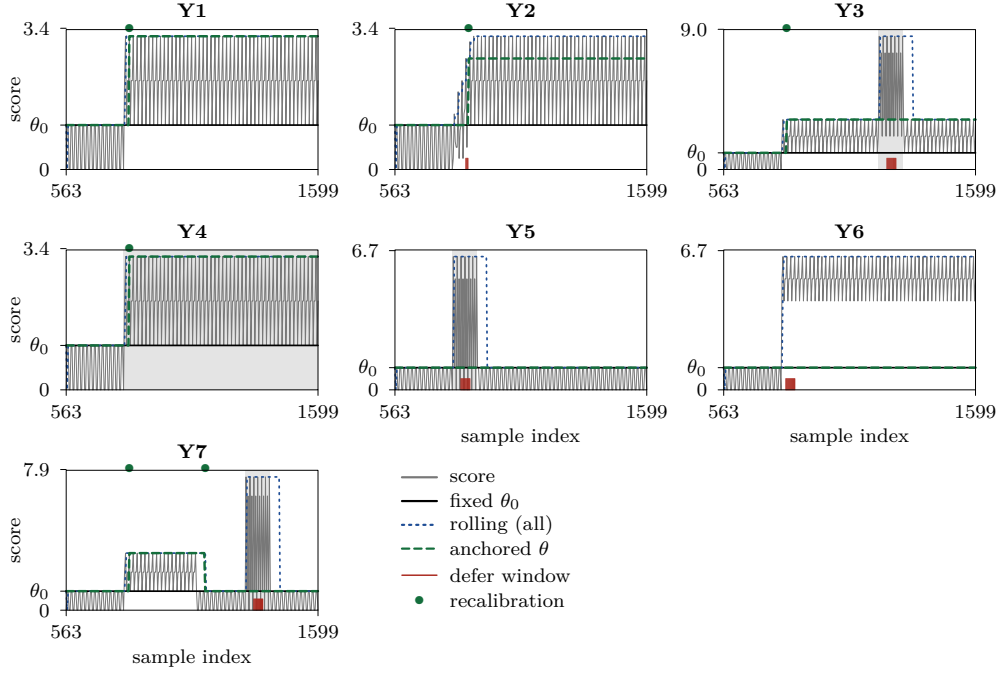


Figure 5: Synthetic cases Y1–Y7: replay score (grey), fixed θ_0 (black), admit-all rolling threshold (blue dotted) and the controller’s threshold (green dashed), with its deferrals (red ticks), recalibrations (green dots) and labelled faults (shaded).

Table 9: Change in each synthetic outcome when one guard component (cap κ , stability ρ or deferral D) is removed, relative to the full controller. Only non-zero changes are shown.

Removed guard	Scen.	$\Delta_{\text{non-ev.}}$	Δ_{defer}	$\Delta_{\text{recal.}}$	$\Delta_{\text{in-ev.}}$
cap κ	Y3	-3	-10	+2	-0.130
	Y5	-3	-10	+2	-0.130
	Y6	-741	-9	+1	–
	Y7	-3	-10	+2	-0.130
stability ρ	Y2	+0	-3	+0	–
deferral D	Y2	+373	-3	-1	–
	Y3	+0	-10	+0	+0.200
	Y5	+0	-10	+0	+0.200
	Y6	+40	-10	+0	–
	Y7	+0	-10	+0	+0.200

licence are not independently verified, so every result is development-only. *Synthetic family*: the stress cases are engineered to test the mechanism and say nothing about real-world benefit.

The strongest opposing conclusion. The strongest honest reading is that these results do *not* establish a useful controller.

- *A simpler rule is quieter.* On the benign cases, admitting every window gives *lower* non-event alert duration than the controller: 12 vs 16 on Y1 and 28 vs 204 on Y2.
- *The controller’s case rests on fault retention alone.* On Y3 its in-event alerted fraction is 0.29 against admit-all’s 0.04, at recall 1/1 for both; fixed alerts 1.000 of the event but at non-event alert duration 521.
- *SKAB gives no evidence for it.* Its rows are identical to fixed on valve1 and valve2.

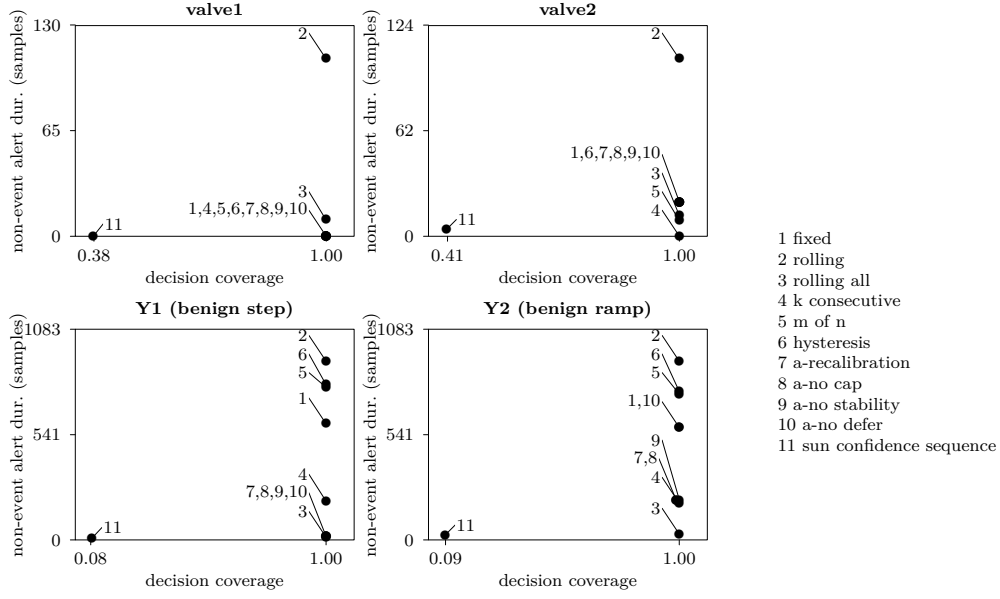


Figure 6: Decision coverage (x) against non-event alert duration in samples (y), one numbered marker per arm (key at right; coincident arms share a label). Arms to the bottom right are quiet *and* decide on every window. Y1 and Y2 contain no fault event.

- *Abstention buys quiet with coverage.* Sun et al.’s arm decides on only 54/143 and 57/140 SKAB windows.

We accept this reading as the Phase-I result: a proved mechanism, a reproducible negative-to-inconclusive development finding, and a predeclared experiment that can discriminate. We do not claim that any policy is superior.

9 Conclusion

We proved that a quantile threshold which learns only from windows it judged normal can never rise, and measured the cost on two SKAB streams: the same single event at $16.2\times$ and $3.9\times$ the fixed threshold’s alert windows. We implemented recalibration as a guarded decision-time action and compared it with seven baselines and three ablations on identical score traces. On SKAB it behaves exactly like the fixed threshold, which passes the no-harm check and shows nothing more. Under synthetic stress it works where designed and fails where any label-free rule must. Whether it helps on *real* benign regime changes is proposed, not claimed, and is settled by the following experiment.

The falsifiable next experiment. **PROPOSED** Replay all arms on common traces for the SKAB streams admitted by a label-independent rule (every `valve1/*.csv` and `valve2/*.csv` other than the two development files, with calibration ending at the declared first labelled row), plus one public stream family with labelled *benign* regime changes. The direction is **refuted**, and that becomes the reported result, if any of these holds:

- on streams where the label-free elevation test fires, the controller does not reduce non-event alert duration versus fixed at equal event recall;
- it loses event recall on any stream where fixed has it;
- Sun et al.’s abstention matches it at equal coverage.

References

- [1] Karan Bhukar, Harshit Kumar, Ruchi Mahindru, Rohan Arora, Seema Nagar, Pooja Aggarwal, and Amit Paradkar. Dynamic alert suppression policy for noise reduction in AIOps. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 178–188, 2024. doi: 10.1145/3639477.3639752.
- [2] Paul Boniol, Ashwin K. Krishna, Marine Bruel, Qinghua Liu, Mingyi Huang, Themis Palpanas, Ruey S. Tsay, Aaron Elmore, Michael J. Franklin, and John Paparrizos. VUS: effective and efficient accuracy measures for time-series anomaly detection. *The VLDB Journal*, 34(3):32, 2025. doi: 10.1007/s00778-025-00907-x.
- [3] Oliver Hennhöfer, Maximilian Kirsch, and Christine Preisach. Conformal anomaly detection in python: Moving beyond heuristic thresholds with nonconform. In *Proceedings of the Fifteenth Symposium on Conformal and Probabilistic Prediction with Applications*, volume 329 of *Proceedings of Machine Learning Research*, pages 613–632. PMLR, 2026. URL <https://proceedings.mlr.press/v329/hennhofer26a.html>.
- [4] Minxuan Hu. A deployment-oriented framework for evaluating streaming integrity alerting under workload constraints. *Expert Systems with Applications*, 333: 134305, 2027. doi: 10.1016/j.eswa.2026.134305. In press (January 2027 issue).
- [5] Iurii D. Katser and Vyacheslav O. Kozitsin. SKAB — skoltech anomaly benchmark. Dataset, 2020. Dataset (data source, not academic evidence).
- [6] Danlei Li, Mingyu Fan, Nirmal-Kumar C. Nair, and Kevin I-Kai Wang. Adaptive drift aware continual learning with pool-based model reuse for time series applications. *Knowledge-Based Systems*, 350:116530, 2026. doi: 10.1016/j.knosys.2026.116530.
- [7] Qinghua Liu and John Paparrizos. The elephant in the room: Towards a reliable time-series anomaly detection benchmark. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024) Datasets and Benchmarks Track*, 2024. doi: 10.52202/079017-3437.
- [8] Lorenzo Perini and Jesse Davis. Unsupervised anomaly detection with rejection. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. doi: 10.52202/075280-3052.
- [9] Abishek Sankararaman, Saumya Gaurang Shah, Balakrishnan Murali Narayanaswamy, and Vikramank Singh. Online anomaly detection in streams with drift and outliers. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 19153–19177. PMLR, 2022. URL <https://proceedings.mlr.press/v162/sankararaman22a.html>.
- [10] Saumya Gaurang Shah, Abishek Sankararaman, Balakrishnan Murali Narayanaswamy, and Vikramank Singh. SEAD: Unsupervised ensemble of streaming anomaly detectors. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 54167–54185. PMLR, 2025. URL <https://proceedings.mlr.press/v267/shah25c.html>.
- [11] Sophia Huiwen Sun, Abishek Sankararaman, and Balakrishnan Murali Narayanaswamy. Online adaptive anomaly thresholding with confidence sequences. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 47105–47132. PMLR, 2024. URL <https://proceedings.mlr.press/v235/sun24h.html>.
- [12] Lijuan Xu, Ziyu Han, Dawei Zhao, Xin Li, Fuqiang Yu, and Chuan Chen. Addressing concept drift in IoT anomaly detection: Drift detection, interpretation, and adaptation. *IEEE Transactions on Sustainable Computing*, 9(6):913–924, 2024. doi: 10.1109/TSUSC.2024.3386667.
- [13] Nancy Xu and Henrik Boström. Conformalized time series anomaly thresholding with latent space features. In *Proceedings of the Symposium on Conformal and Probabilistic Prediction with Applications*, volume 329 of *Proceedings of Machine Learning Research*, pages 676–688. PMLR, 2026. URL <https://proceedings.mlr.press/v329/xu26a.html>.
- [14] Licheng Yang, Yu Yao, Daoqing Yang, Wei Yang, and Yuming Hao. A generalizable anomaly detection framework with dynamic concept drift suppression for non-stationary time series. *Knowledge-Based Systems*, 337:115380, 2026. doi: 10.1016/j.knosys.2026.115380.